

Rekurze I

IB111 ZÁKLADY PROGRAMOVÁNÍ

Tomáš Vojnar (na základě slajdů Nikoly Beneše)

14. listopadu 2025

Co je rekurze?

- **Rekurze:** definice nějakého konceptu pomocí téhož konceptu.
 - Podprogram, který volá sám sebe.
 - Datová struktura, která jako podstrukturu obsahuje tutéž strukturu.

Už jste něco podobného určitě viděli:

Co je rekurze?

- **Rekurze:** definice nějakého konceptu pomocí téhož konceptu.
 - Podprogram, který volá sám sebe.
 - Datová struktura, která jako podstrukturu obsahuje tutěž strukturu.

Už jste něco podobného určitě viděli:

- Induktivní definice,
 - např. přirozených čísel, formulí výrokové logiky.
- Definice faktoriálu.
- Rekurze ve funkcionálním programování,
 - pokud máte zapsáno IB015.
- Fraktály.
- Pravděpodobně i jinde...

Rekurze v programování

- Je hlavně *způsob přemýšlení o řešení problémů*.
- Při vymýšlení algoritmu můžeme předpokládat, že menší instance někdo¹ vyřeší za nás
 - a tedy se o ně nemusíme starat,
 - musíme ale dodržet *pravidla*.
- Hodí se zejména pro práci se složitějšími datovými strukturami,
 - ale má i jiné užitečné aplikace.

¹Skřítkci, víly, *recursion fairy*, jak je libo...

1. Dostatečně jednoduché případy, tzv. bazové případy, musíme vyřešit přímo.

1. Dostatečně jednoduché případy, tzv. bazové případy, musíme vyřešit **přímo**.
2. Rekurzivně řešené pod-případy musí být v nějakém smyslu **jednodušší**, než je aktuální případ.

Co není rekurze?

Rekurze **není** definice kruhem.

Co není rekurze?

Rekurze **není** definice kruhem. Rekurze je spirála matrjoška.



Source: wiki.

Faktoriál rekurzivně

```
def fact(num: int) -> int:
    if num == 0:
        return 1

    return num * fact(num - 1)
```

- Dodržuje dvě pravidla (správné) rekurze?

Volání (rekurzivních) podprogramů

- Při volání podprogramů se vytváří **zásobník volání** (*call stack*), který obsahuje:
 - lokální proměnné, včetně parametrů, a vazby na objekty s jejich hodnotami,
 - informaci o tom, kde pokračovat v přerušeném výpočtu.
- Pro ilustraci: <http://pythontutor.com>
- U rekurzivních podprogramů může obsahovat záznamy o více rozpracovaných aktivacích daného podprogramu.
 - Buď přímo za sebou nebo proložené s dalšími podprogramy.

Vstup: kladné celé číslo N.

Napište proceduru, která vypíše čísla od 1 do N *bez použití cyklů*.

Vstup: kladné celé číslo N.

Napište proceduru, která vypíše čísla od 1 do N *bez použití cyklů*.

```
def sequence(num: int) -> None:
    if num > 1:
        sequence(num - 1)
    print(num)
```

Vstup: kladné celé číslo N .

Napište *čistou funkci*, která vrátí seznam čísel od 1 do N *bez použití cyklů*.

Vstup: kladné celé číslo N.

Napište *čistou funkci*, která vrátí seznam čísel od 1 do N *bez použití cyklů*.

```
def seq_pure1(num: int) -> list[int]:  
    if num == 0:  
        return []  
    return seq_pure1(num - 1) + [num]
```

Jakou má tato funkce složitost?

Vstup: kladné celé číslo N.

Napište *čistou funkci*, která vrátí seznam čísel od 1 do N *bez použití cyklů*.

```
def seq_pure1(num: int) -> list[int]:  
    if num == 0:  
        return []  
    return seq_pure1(num - 1) + [num]
```

Jakou má tato funkce složitost? *Kvadratickou vůči N!*

Vstup: kladné celé číslo N .

Napište *čistou funkci*, která vrátí seznam čísel od 1 do N *bez použití cyklů*. Funkce musí mít *lineární složitost vůči N* .

Vstup: kladné celé číslo N.

Napište *čistou funkci*, která vrátí seznam čísel od 1 do N *bez použití cyklů*. Funkce musí mít *lineární složitost vůči N*.

```
def seq_pure2(num: int) -> list[int]:  
    if num == 0:  
        return []  
    result = seq_pure2(num - 1)  
    result.append(num)  
    return result
```

```
def mystery_rec(num: int, result: list[int]) -> None:
    result.append(num)
    if num > 1:
        mystery_rec(num - 1, result)
    result.append(-num)

def mystery(num: int) -> list[int]:
    result: list[int] = []
    mystery_rec(num, result)
    return result
```

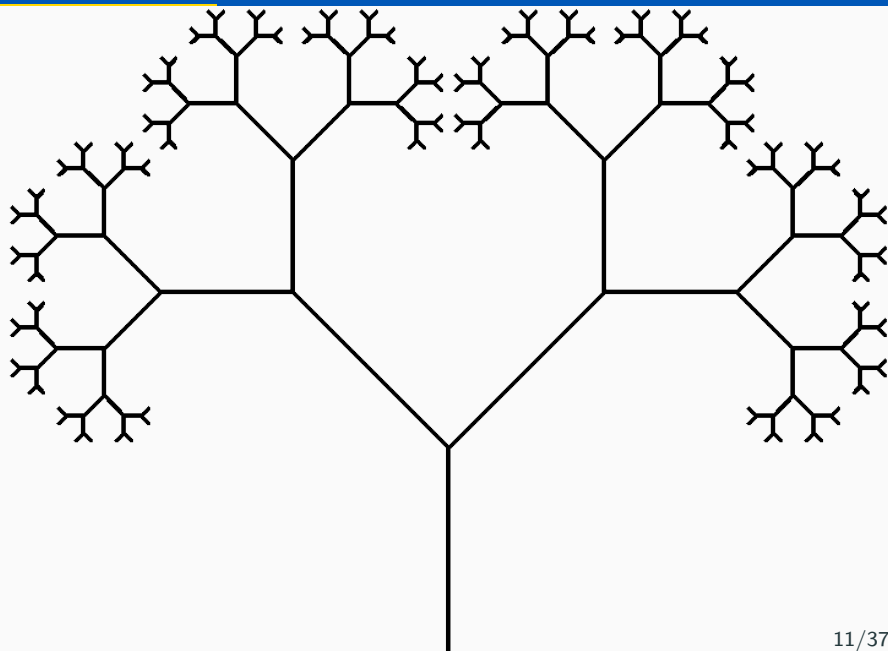
- Co dělá funkce `mystery`?
- Je funkce `mystery` čistá?

Rekurzivní funkce se nemusí volat přímo, ale i přes jinou funkci.

```
def even(num: int) -> bool:
    if num == 0:
        return True
    return odd(num - 1)
```

```
def odd(num: int) -> bool:
    if num == 0:
        return False
    return even(num - 1)
```

Rekurzivní stromeček



Jak nakreslit rekurzivní stromeček?

- Nakreslit kmen.
 - Nakreslit dva menší stromečky (pootočené).
 - Vrátit se zpátky.
-
- Kdo za nás nakreslí ty menší stromečky?

Jak nakreslit rekurzivní stromeček?

- Nakreslit kmen.
- Nakreslit dva menší stromečky (pootočené).
- Vrátit se zpátky.
- Kdo za nás nakreslí ty menší stromečky?
 - Skřítkci, víly, *recursion fairy*, rekurze!
- Dodržujeme pravidla?

Jak nakreslit rekurzivní stromeček?

- Nakreslit kmen.
- Nakreslit dva menší stromečky (pootočené).
- Vrátit se zpátky.

- Kdo za nás nakreslí ty menší stromečky?
 - Skřítkci, víly, *recursion fairy*, rekurze!

- Dodržujeme pravidla?
 - První pravidlo:

Jak nakreslit rekurzivní stromeček?

- Nakreslit kmen.
- Nakreslit dva menší stromečky (pootočené).
- Vrátit se zpátky.

- Kdo za nás nakreslí ty menší stromečky?
 - Skřítkci, víly, *recursion fairy*, rekurze!

- Dodržujeme pravidla?
 - První pravidlo: Ne...

Jak nakreslit rekurzivní stromeček?

- Nakreslit kmen.
- Nakreslit dva menší stromečky (pootočené).
- Vrátit se zpátky.

- Kdo za nás nakreslí ty menší stromečky?
 - Skřítcí, víly, *recursion fairy*, rekurze!

- Dodržujeme pravidla?
 - První pravidlo: Ne...
 - Kdy skončit?

Jak nakreslit rekurzivní stromeček?

- Nakreslit kmen.
- Nakreslit dva menší stromečky (pootočené).
- Vrátit se zpátky.
- Kdo za nás nakreslí ty menší stromečky?
 - Skřítcí, víly, *recursion fairy*, rekurze!
- Dodržujeme pravidla?
 - První pravidlo: Ne...
 - Kdy skončit? Když už je kmen dostatečně krátký.
 - Druhé pravidlo: OK.

```
def tree(length: float) -> None:
    forward(length)
    if length > 10:
        left(45)
        tree(0.6 * length)
        right(90)
        tree(0.6 * length)
        left(45)
    backward(length)
```

```
def wrong1(num: int) -> int:  
    return num * wrong1(num - 1)
```

```
def wrong2(num: int) -> int:  
    if num <= 1:  
        return num  
  
    return 1 + wrong2(num)
```

- Co se stane?

```
def wrong3(num: int) -> int:
    if num > 1000:
        return 1000

    return 1 + wrong3(num - 1)
```

- Porušujeme zde pravidlo 2?

```
def wrong3(num: int) -> int:
    if num > 1000:
        return 1000

    return 1 + wrong3(num - 1)
```

- Porušujeme zde pravidlo 2?
 - ANO.
 - Jednodušší nemusí nutně znamenat menší!
 - Jednodušší znamená *blíže k situaci, kterou řešíme přímo.*

```
def wrong4(num: float) -> float:  
    if num < 0:  
        return 0  
  
    return 1 + wrong4(num / 2)
```

- Porušujeme zde pravidlo 2?

```
def wrong4(num: float) -> float:
    if num < 0:
        return 0

    return 1 + wrong4(num / 2)
```

- Porušujeme zde pravidlo 2?
 - ANO.
 - Pro některé vstupy v rekurzivním volání nezjednodušujeme.
- Co kdyby místo `<` bylo `<=`?

```
def wrong4(num: float) -> float:
    if num < 0:
        return 0

    return 1 + wrong4(num / 2)
```

- Porušujeme zde pravidlo 2?
 - ANO.
 - Pro některé vstupy v rekurzivním volání nezjednodušíme.
- Co kdyby místo `<` bylo `<=`?
 - Stále porušujeme (alespoň na úrovni myšlenky algoritmu).

Upřesnění pravidla 2: Musí být nějaký **minimální krok**, alespoň o který se při zjednodušování vždy posuneme.

- Velikost zásobníku volání je typicky nějak omezena.
 - V Pythonu implicitně na 1000 volání, dá se změnit.
- Překročení limitu zásobníku volání je chyba.
 - V Pythonu `RecursionError`.
 - Jinde též `stack overflow`.
- Znamená to, že nemáme při programování používat rekurzi?

- Velikost zásobníku volání je typicky nějak omezena.
 - V Pythonu implicitně na 1000 volání, dá se změnit.
- Překročení limitu zásobníku volání je chyba.
 - V Pythonu `RecursionError`.
 - Jinde též stack overflow.
- Znamená to, že nemáme při programování používat rekurzi?
- NE! Znamená to, že ji máme používat *rozumně*.
- Nezapomeňte, že rekurze je i způsob *přemýšlení*.

Obecně

- Každý rekurzivní program je možno přepsat jako iterativní za pomoci zásobníku.
- **Explicitní zásobník zde simuluje zásobník volání podprogramů:**
 - lokální proměnné a jejich vazby,
 - kde pokračovat.
- Mechanický přepis se často, byť ne vždy, dá zjednodušit.
 - Např. mechanický převod faktoriálu x jednoduchý cyklus.

Obecně

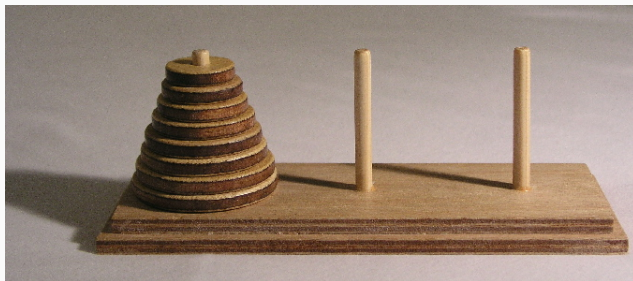
- Každý rekurzivní program je možno přepsat jako iterativní za pomoci zásobníku.
- **Explicitní zásobník zde simuluje zásobník volání podprogramů:**
 - lokální proměnné a jejich vazby,
 - kde pokračovat.
- Mechanický přepis se často, byť ne vždy, dá zjednodušit.
 - Např. mechanický převod faktoriálu x jednoduchý cyklus.

Koncová rekurze (tail recursion)

- Rekurzivní volání je to poslední, co se v podprogramu stane.
- Lze snadno nahradit cyklem, bez použití zásobníku.
- Některé kompilátory/interprety toto optimalizují samy.
 - Python ne (bohužel).

Hanojské věže

- Spíše pro zajímavost, ukázka toho, jak „mocná“ je rekurze.
- Pravidla:
 - vždy se smí přesouvat jen jeden disk,
 - nesmí se položit větší disk na menší.



Source: wiki.

Chceme přesunout N nejmenších disků:

- Pokud je $N = 1$:

Chceme přesunout N nejmenších disků:

- Pokud je $N = 1$:
 - Přesuneme disk přímo (je nejmenší, nemáme žádná omezení).
- Jinak:

Chceme přesunout N nejmenších disků:

- Pokud je $N = 1$:
 - Přesuneme disk přímo (je nejmenší, nemáme žádná omezení).
- Jinak:
 - Někdo za nás přesune $N-1$ nejmenších disků na pomocné místo.
 - Přesuneme N -tý disk přímo na cílové místo.
 - Někdo za nás přesune $N-1$ nejmenších disků z pomocného místa na cílové.

A to je vše.

- Kdo za nás bude přesouvat ty menší věže?

Chceme přesunout N nejmenších disků:

- Pokud je $N = 1$:
 - Přesuneme disk přímo (je nejmenší, nemáme žádná omezení).
- Jinak:
 - Někdo za nás přesune $N-1$ nejmenších disků na pomocné místo.
 - Přesuneme N -tý disk přímo na cílové místo.
 - Někdo za nás přesune $N-1$ nejmenších disků z pomocného místa na cílové.

A to je vše.

- Kdo za nás bude přesouvat ty menší věže? Rekurze.

```
def hanoi(num: int, start: str, end: str, aux: str) \
    -> None:
    if num == 1:
        print(start, "->", end)
    else:
        hanoi(num - 1, start, aux, end)
        print(start, "->", end)
        hanoi(num - 1, aux, end, start)
```

```
def hanoi(num: int, start: str, end: str, aux: str) \
    -> None:
    if num == 1:
        print(start, "->", end)
    else:
        hanoi(num - 1, start, aux, end)
        print(start, "->", end)
        hanoi(num - 1, aux, end, start)
```

- Neopakujeme se zbytečně?

- (O něco) lepší nápad – základ bude 0, ne 1.
 - Co je potřeba udělat pro vstup 0?

- (O něco) lepší nápad – základ bude 0, ne 1.
 - Co je potřeba udělat pro vstup 0? NIC!

- (O něco) lepší nápad – základ bude 0, ne 1.
 - Co je potřeba udělat pro vstup 0? NIC!

```
def hanoi_better(num: int, start: str,
                 end: str, aux: str) -> None:
    if num > 0:
        hanoi_better(num - 1, start, aux, end)
        print(start, "->", end)
        hanoi_better(num - 1, aux, end, start)
```

```
State = dict[str, list[int]]
```

```
def move(source: str, destination: str,  
        state: State) -> None:  
    print("Move", source, "->", destination)  
    state[destination].append(state[source].pop())  
    print(state)
```

```
def hanoi_solve(num: int, start: str,  
               end: str, aux: str,  
               state: State) -> None:  
    if num > 0:  
        hanoi_solve(num - 1, start, aux, end, state)  
        move(start, end, state)  
        hanoi_solve(num - 1, aux, end, start, state)
```

K rozmyšlení na doma (částečně probereme na další přednášce):

- Implementace ve formě *čisté funkce*,
 - návratová hodnota – seznam dvojic (odkud, kam).
 - Jak implementovat efektivně?
- Iterativní řešení (se zásobníkem).
- Iterativní řešení *bez zásobníku* – výraznamně těžší.

Sierpińského trojúhelník



- Příklad tzv. *fraktálu*:
 - rekurzivně definované útvary, podobně jako „stromček“,
 - opakující se struktura při libovolném přiblížení.
- Nakreslíme pomocí želvy (a rekurze) – bez výplně.

Pro extrémně zvědavé: Tento obrazec souvisí s Hanojskými věžemi.

```
def sierpinski(steps: int, length: int) -> None:
    if steps == 1:
        triangle(length)
    else:
        for _ in range(3):
            sierpinski(steps - 1, length)
            forward(2 ** (steps - 1) * length)
            left(120)
```

- I tady bychom mohli jako základ vzít 0 místo 1.
 - Zkuste si rozmyslet, co by to znamenalo.

Rodokmen

- Chceme vytvořit rodokmen.
- Každý objekt bude reprezentovat člena rodiny.
- Každý člen rodiny má:
 - jméno,
 - seznam dětí,
 - rodiče.
- Pro zjednodušení:
 - každý má v rodokmenu jen jednoho rodiče,
 - typické pro rodokmeny s jedním prapředkem.

```
class Person:
    def __init__(self, name: str):
        self.name = name
        self.parent: Person | None = None
        self.children: list[Person] = []

    def add_child(self, child: 'Person') -> None:
        self.children.append(child)
        child.parent = self
```

Poznámka: Všimněte si dopředné reference na typ Person v typové anotaci metody add_child.

```
# The Simpsons
```

```
abe = Person("Abraham Simpson")
```

```
homer = Person("Homer Simpson")
```

```
abe.add_child(homer)
```

```
homer.add_child(Person("Bart Simpson"))
```

```
homer.add_child(Person("Lisa Simpson"))
```

```
homer.add_child(Person("Maggie Simpson"))
```

```
bart = homer.children[0]
```

```
lisa = homer.children[1]
```

```
maggie = homer.children[2]
```

- Chceme predikát `siblings(person_a, person_b)`, který zjistí, zda jsou `person_a`, `person_b` sourozenci.

- Chceme predikát `siblings(person_a, person_b)`, který zjistí, zda jsou `person_a`, `person_b` sourozenci.

```
def siblings(person_a: Person,  
             person_b: Person) -> bool:  
    return person_a.parent is not None \  
           and person_a.parent == person_b.parent
```

- Chceme najít nejstaršího předka v rodokmenu
- Jak na to?

- Chceme najít nejstaršího předka v rodokmenu
- Jak na to?

```
def oldest_ancestor1(person: Person) -> Person:  
    while person.parent is not None:  
        person = person.parent  
    return person
```

- A co to zkusit rekurzivně?

- Chceme najít nejstaršího předka v rodokmenu
- Jak na to?

```
def oldest_ancestor1(person: Person) -> Person:
    while person.parent is not None:
        person = person.parent
    return person
```

- A co to zkusit rekurzivně?

```
def oldest_ancestor2(person: Person) -> Person:
    if person.parent is None:
        return person
    return oldest_ancestor2(person.parent)
```

- Chceme počet všech (přímých i nepřímých) potomků dané osoby.
 - Rekurzivní řešení je jednoduché,
 - iterativní je o něco složitější.

- Chceme počet všech (přímých i nepřímých) potomků dané osoby.
 - Rekurzivní řešení je jednoduché,
 - iterativní je o něco složitější.

```
def count_offspring(person: Person) -> int:  
    count = 0  
    for child in person.children:  
        count += 1 + count_offspring(child)  
    return count
```

- Chceme rodokmen vypsát (s vyznačením struktury).
- Máme připravenou proceduru `print_with_indent`, která vypíše zadaný řetězec se zadaným odsazením.

- Chceme rodokmen vypsat (s vyznačením struktury).
- Máme připravenou proceduru `print_with_indent`, která vypíše zadaný řetězec se zadaným odsazením.

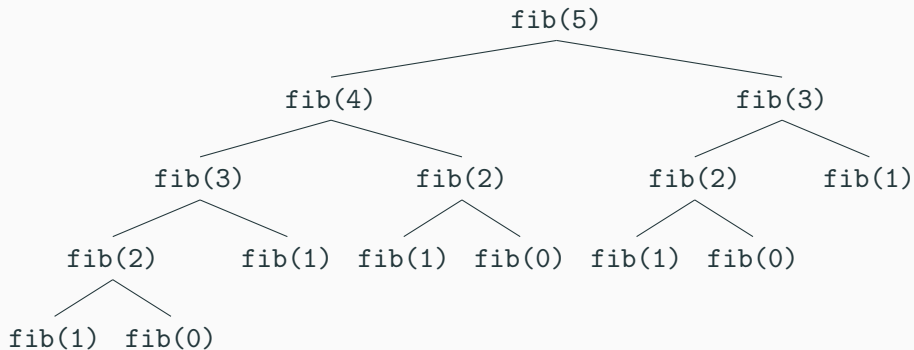
```
def draw_family_tree(person: Person) -> None:  
    draw_family_tree_aux(person, 0)
```

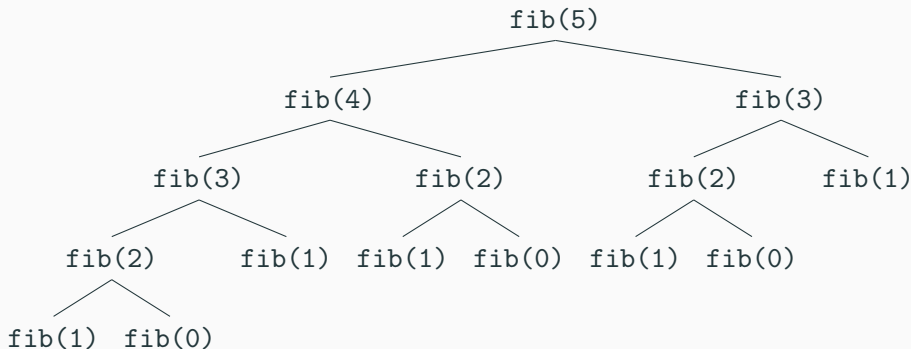
```
def draw_family_tree_aux(person: Person,  
                           indent: int) -> None:  
    print_with_indent(indent, person.name)  
    for child in person.children:  
        draw_family_tree_aux(child, indent + 4)
```

- Co je špatně v následujícím řešení?

```
def fib(num: int) -> int:
    if num < 2:
        return num
    return fib(num - 1) + fib(num - 2)
```

Strom rekurze





- Některé (skoro všechny) výpočty se zbytečně opakují.
- *Exponenciální* složitost vůči hodnotě num.
- Řešení: memoizace (zapamatování mezivýsledků) či zde jednoduchý cyklus.

- Euklidův algoritmus (tail rekurze) – viz přednáška 3,
- binární vyhledávání (tail rekurze),
- řadicí algoritmy:
 - mergesort,
 - quicksort,
- generování permutací, kombinací,
- řešení problémů splnění omezení „hrubou silou“ (tzv. backtracking),
- prohledávání grafu (do hloubky),
- gramatiky – syntaktická analýza,
- práce s rekurzivními datovými strukturami,
- ...

Trénink rekurze: <https://www.umimeprogramovat.cz/robotanik>

- velmi jednoduché „příkazy“ pro robota,
- těžší příklady založeny (čistě) na rekurzi,
- vizualizace průběhu výpočtu a rekurzivních volání.